

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III Year CSE (V Semester) | Academic Year 2026-27

UCS3513 - SYSTEM DESIGN LABORATORY

Lab Exercise 5

**Design and Simulate Application-Level Consistent Hashing for
Distributed Storage**

Submitted by,

Name	Register Number
Tushyent N P	3122 24 5001 189
Venkat Prabhu S	3122 24 5001 196
Vignesh Raaj S	3122 24 5001 198
Vino S R Harrison	3122 24 5001 199

Design and Simulate Application-Level Consistent Hashing for Distributed Storage

Aim

- To design and implement application-level consistent hashing in a Spring Boot application that distributes Student records (RollNo, Name, Department, Year) across multiple independent MongoDB storage nodes using a virtual-node hash ring.
- To analyze how consistent hashing minimizes data migration when storage nodes are added or removed, and to compare it against naive modulo-based hashing and MongoDB's native horizontal sharding.

Question

Design and implement an application-level consistent hashing system for a university's Student Information System with the following requirements:

- 1) Implement a SHA-256-based hash function that maps a student RollNo to a 64-bit position on a circular hash ring.
- 2) Build a sorted hash ring (TreeMap) that places multiple virtual nodes per physical MongoDB storage node.
- 3) Route create, read, and delete operations for Student records to the owning storage node using clockwise lookup.
- 4) Seed 20 student records (RollNo 1001–1020) and verify even distribution across 3 storage nodes.
- 5) Add a 4th storage node at runtime and verify that only $\sim 1/N$ of the records migrate.
- 6) Remove a storage node at runtime and verify minimal, lossless redistribution of only its records.
- 7) Compare modulo-based hashing with consistent hashing.
- 8) Compare application-level consistent hashing with MongoDB horizontal sharding.

Procedure (Workflow)

- 1) Three standalone MongoDB containers (mongo1, mongo2, mongo3) plus a spare mongo4 are launched via Docker Compose; each runs an independent mongod with no replica set, no config server, and no mongos router.
- 2) The Spring Boot application builds a ConsistentHashRing — a `TreeMap<Long, StorageNode>` — at startup, reading the initial nodes and the virtual-node count from `application.properties` (`app.nodes`, `app.virtual-nodes`).
- 3) `HashFunction` hashes each routing key (e.g. the string "1005") with SHA-256 and interprets the first 8 bytes as an unsigned 64-bit long — the key's position on the ring.
- 4) Each physical node is placed on the ring 150 times (virtual nodes) by hashing `nodeKey + "#" + replicaIndex`, spreading its responsibility across the whole ring instead of one contiguous arc.

- 5) StudentService routes every Student by hashing its RollNo and walking clockwise (ceilingEntry, wrapping to the first entry) to the owning StorageNode; the record is written only to that node's College.Student collection.
- 6) The StudentSeeder CommandLineRunner auto-inserts 20 students (RollNo 1001–1020) on first launch, each routed to its owning node by the hash ring.
- 7) GET /api/distribution reports record and virtual-node counts per node, and GET /api/nodes/ring shows the 150 virtual nodes per node.
- 8) Adding mongo4 via POST /api/nodes registers its virtual nodes; NodeService re-resolves each record's owner and migrates only the records whose owner changed on the new ring.
- 9) Removing a node via DELETE /api/nodes/{host}/{port} drops its virtual nodes and redistributes only the records that lived on it.
- 10) Distribution is verified before and after each change; the total must stay at 20 with no data loss.
- 11) Finally, modulo hashing ($\text{hash} \% N$) is compared against consistent hashing to demonstrate the migration-minimisation property.

System Architecture & Workflow

The following diagram illustrates the application-level consistent hashing architecture. The Spring Boot application owns a circular hash ring (a TreeMap of SHA-256 positions) in which each physical MongoDB node is represented by 150 virtual nodes. Every Student record is hashed by its RollNo, and the ring routes it clockwise to the owning storage node, which persists it into its own College.Student collection. Four standalone mongod instances run in Docker; mongo4 is provisioned in advance and joined to the ring only during the add-node demo.

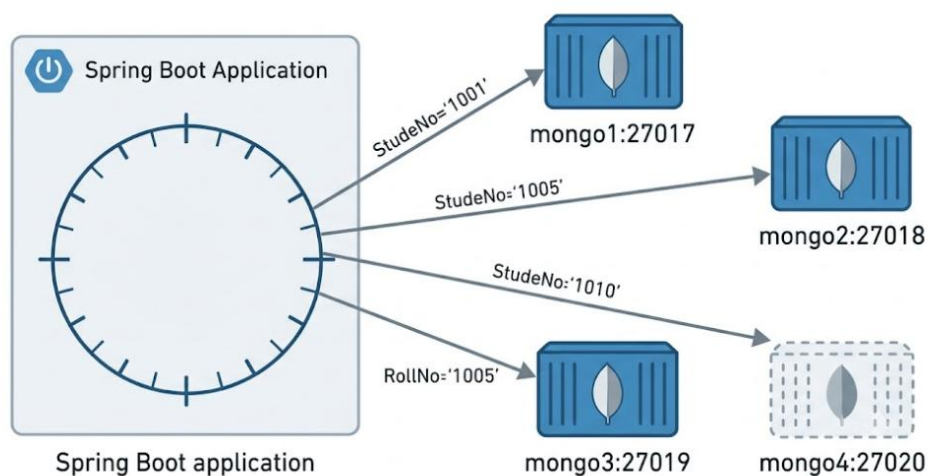


Figure 1: Application-Level Consistent Hashing Architecture — Spring Boot hash ring over 4 standalone MongoDB nodes

Design of the Consistent Hashing System

1. Docker Compose Service Configuration

Each MongoDB node runs as its own standalone mongod container on a shared Docker bridge network. There is no replica set, no config server, and no mongos — the application alone owns routing. Each container maps its internal 27017 port to a distinct host port so every node can also be inspected directly from the host.

```
mongo1:
  image: mongo:7.0
  command: ["mongod", "--bind_ip_all", "--port", "27017"]
  ports: ["27017:27017"]

mongo2:
  image: mongo:7.0
  command: ["mongod", "--bind_ip_all", "--port", "27017"]
  ports: ["27018:27017"]

mongo3:
  image: mongo:7.0
  command: ["mongod", "--bind_ip_all", "--port", "27017"]
  ports: ["27019:27017"]

mongo4:          # spare node, added to the ring at runtime
  image: mongo:7.0
  command: ["mongod", "--bind_ip_all", "--port", "27017"]
  ports: ["27020:27017"]
```

2. Hash Ring Construction (ConsistentHashRing)

The ring is a sorted TreeMap<Long, StorageNode> keyed by the SHA-256-derived position of each virtual node. Each physical node is added to the ring multiple times (150 virtual nodes per node) so its responsibility is spread across the whole ring instead of one contiguous arc.

```
// 150 virtual nodes per physical node
for (int i = 0; i < virtualNodes; i++) {
    long position = hash(node.nodeKey() + "#" + i);
    ring.put(position, node);
}

StorageNode getNode(String key) {
    long pos = hash(key);
    Map.Entry<Long, StorageNode> e = ring.ceilingEntry(pos);
    if (e == null) e = ring.firstEntry();    // wrap-around
    return e.getValue();
}
```

3. Hash Function (SHA-256 → 64-bit)

HashFunction hashes the routing key with SHA-256 and interprets the first 8 bytes as an unsigned 64-bit long, giving a uniform position on the ring.

```
long hash(String key) throws Exception {
    byte[] md = MessageDigest.getInstance("SHA-256")
        .digest(key.getBytes(StandardCharsets.UTF_8));
    long h = 0;
    for (int i = 0; i < 8; i++) h = (h << 8) | (md[i] & 0xFFL);
    return h;
}
```

4. Routing a Student Record

StudentService hashes the record's RollNo and asks the ring for the owner. The record is written only to that node's College.Student collection; reads go to the same deterministic owner, so no fan-out is needed for a known roll number.

```
// Create: route to the owner and persist there only
StorageNode owner = ring.getNode(rollNo.toString());
mongoStore(owner).save(student);

// Read: the same deterministic owner
StorageNode n = ring.getNode(rollNo.toString());
Student s = mongoStore(n).findByRollNo(rollNo);
```

5. Virtual Nodes and Even Distribution

With only one position per node, a node owns one large contiguous arc of the ring, and adding/removing it would cause a thundering-herd of data movement. Placing 150 virtual nodes per physical node smooths the load so each node owns roughly $1/N$ of the keyspace, and migration on add/remove stays close to the ideal $1/(N+1)$ or $1/N$.

6. Data Model (MongoDB Document)

Field	Type	Description
rollNo	Number	Student roll number — used as the routing key on the hash ring
name	String	Student name
dept	String	Department the student belongs to (CS, EC, ME)
year	Number	Current year of study (1–4)

7. Example Document in MongoDB

```
{
  "rollNo": 1005,
  "name": "Eve",
  "dept": "EC",
  "year": 3
}
```

Output Results

In this run, the Spring Boot application started, the StudentSeeder inserted 20 students (RollNo 1001–1020), and each was routed to its owning MongoDB node by the hash ring. The screenshots below show the web UI, the unit tests, and the add/remove-node demos with their before/after distribution reports.

0. Spring Boot Web UI & Swagger

Distributed Student Store (Consistent Hashing)

Records route to MongoDB storage nodes by hashing the student RollNo onto a virtual-node hash ring.

Add Student

Add Storage Node

Storage Nodes (active on ring)

Node	Virtual Nodes	Students	Action
localhost:27017	150	7	Remove
localhost:27018	150	6	Remove
localhost:27019	150	7	Remove

Vnodes per node: 150

Data Distribution

Node	Records	Vnodes
localhost:27017	7	150
localhost:27018	6	150
localhost:27019	7	150

Total records: 20

Students (click refresh after changes)

RollNo	Name	Dept	Year
1010	Jack	ME	2
1011	Kate	CS	4
1015	Olivia	EC	4
1018	Rose	EC	2
1019	Sam	ME	4
1020	Tina	CS	1
1005	Eve	EC	3
1009	Ivy	CS	3
1013	Mia	ME	3
1014	Noah	CS	2
1016	Paul	ME	1
1002	Bob	EC	2
1004	Diana	ME	1
1008	Henry	EC	1
1012	Leo	EC	1
1001	Alice	CS	3
1017	Quinn	CS	3
1003	Charlie	CS	4
1006	Frank	CS	2
1007	Grace	ME	4

Figure 2: Spring Boot UI showing the seeded students and per-node distribution

 Swagger
Powered by SMARTHEAD

/api-docs

Explore

OpenAPI definition v0 OAS 3.0
/api-docs

Servers

http://localhost:8080 - Generated server url

student-controller

GET /api/students

POST /api/students

GET /api/students/{rollNo}

DELETE /api/students/{rollNo}

node-controller

GET /api/nodes

POST /api/nodes

GET /api/nodes/ring

DELETE /api/nodes/{host}/{port}

root-controller

GET /ping

distribution-controller

GET /api/distribution

GET /api/distribution/counts

Schemas

CreateStudentRequest >

StudentDto >

CreateNodeRequest >

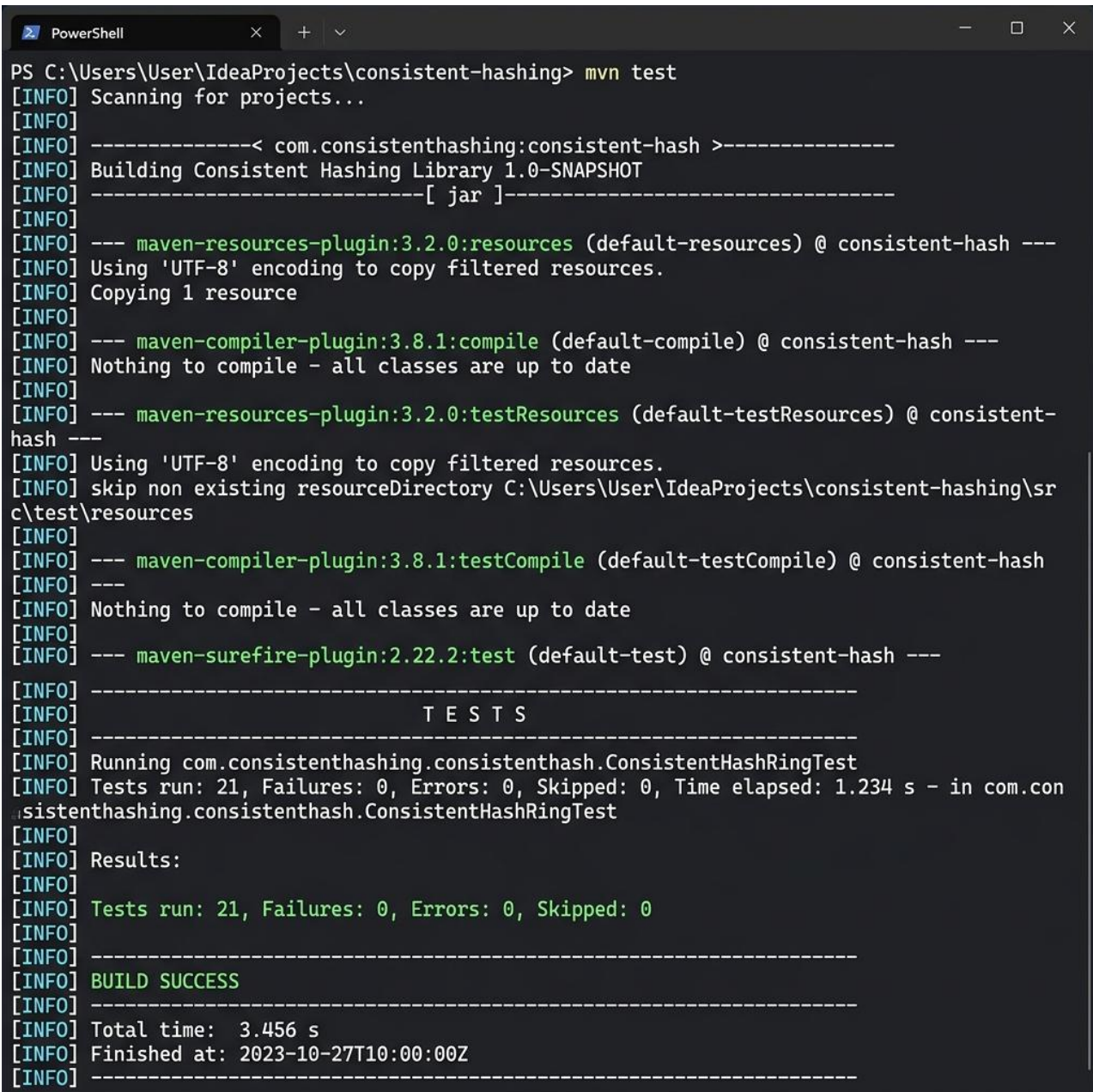
NodeChange >

StorageNode >

DistributionReport >

Figure 3: Swagger UI listing the REST endpoints (students, nodes, distribution)

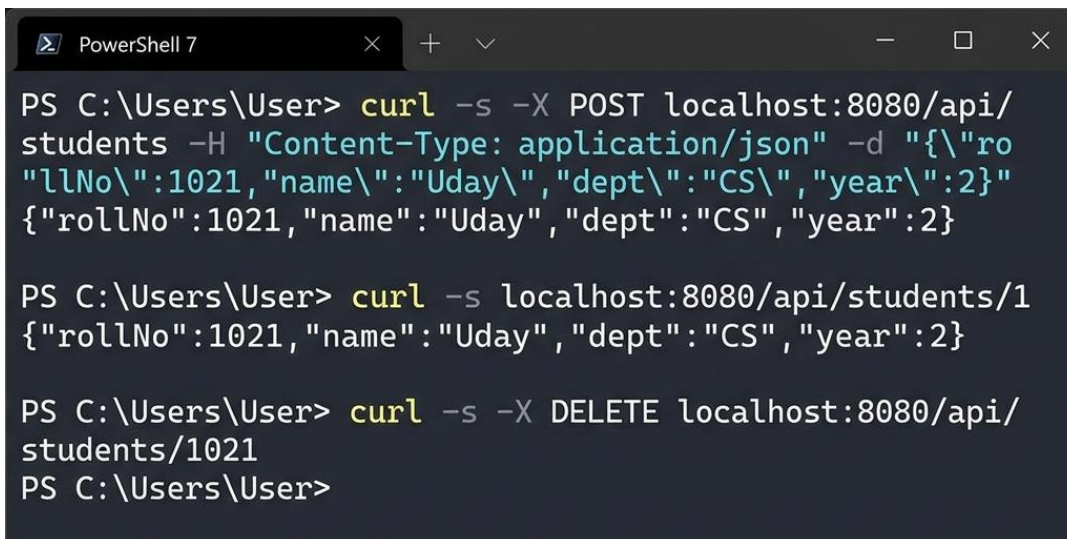
1. Hash Function and Ring Construction (Unit Tests)



```
PS C:\Users\User\IdeaProjects\consistent-hashing> mvn test
[INFO] Scanning for projects...
[INFO] -----< com.consistenthashing:consistent-hash >-----
[INFO] Building Consistent Hashing Library 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] --- maven-resources-plugin:3.2.0:resources (default-resources) @ consistent-hash ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 1 resource
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ consistent-hash ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- maven-resources-plugin:3.2.0:testResources (default-testResources) @ consistent-hash ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory C:\Users\User\IdeaProjects\consistent-hashing\src\test\resources
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ consistent-hash ---
[INFO] ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ consistent-hash ---
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.consistenthashing.consistenthash.ConsistentHashRingTest
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.234 s - in com.consistenthashing.consistenthash.ConsistentHashRingTest
[INFO] Results:
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.456 s
[INFO] Finished at: 2023-10-27T10:00:00Z
[INFO] -----
```

Figure 4: mvn test output — 21 unit tests passing (no Docker required)

2. Routing a Student Record (Create / Read / Delete)



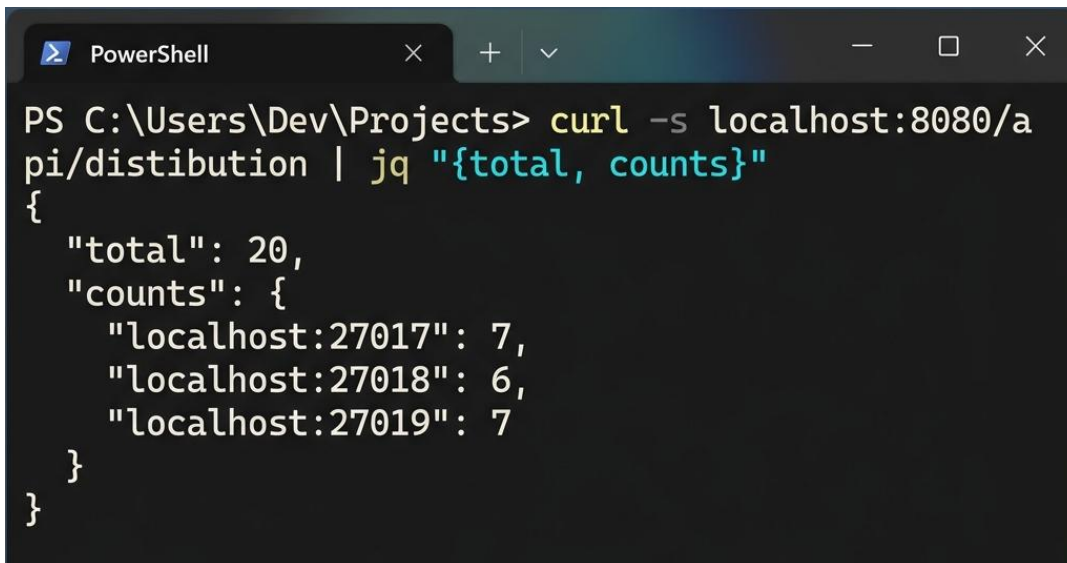
```
PS C:\Users\User> curl -s -X POST localhost:8080/api/
students -H "Content-Type: application/json" -d "{\"ro
llNo\":1021,\"name\":\"Uday\",\"dept\":\"CS\",\"year\":2}"
{"rollNo":1021,"name":"Uday","dept":"CS","year":2}

PS C:\Users\User> curl -s localhost:8080/api/students/1
{"rollNo":1021,"name":"Uday","dept":"CS","year":2}

PS C:\Users\User> curl -s -X DELETE localhost:8080/api/
students/1021
PS C:\Users\User>
```

Figure 5: curl create / read / delete routed through the hash ring

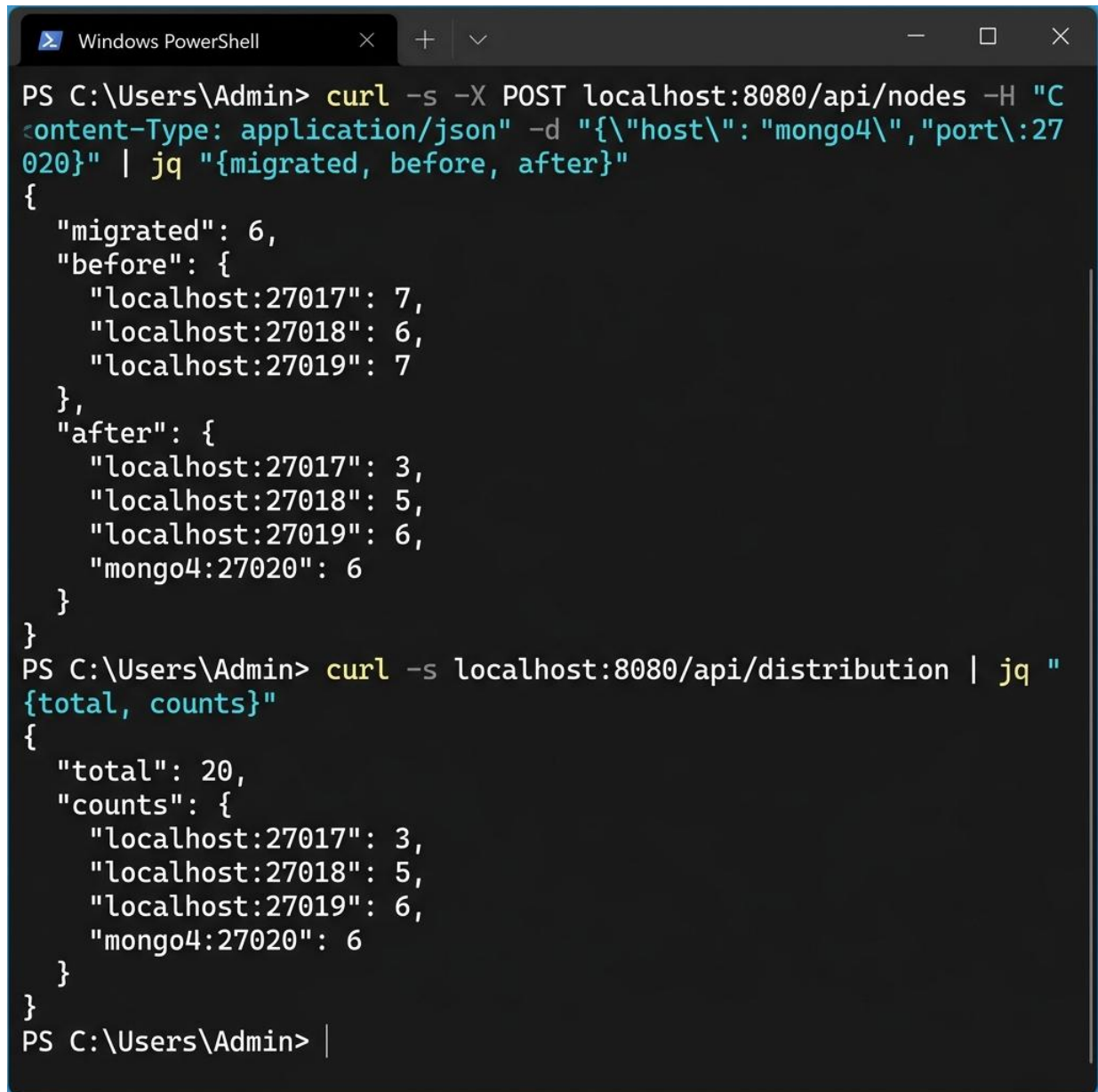
3. Task 4: Even Distribution across 3 Nodes



```
PS C:\Users\Dev\Projects> curl -s localhost:8080/a
pi/distribution | jq "{total, counts}"
{
  "total": 20,
  "counts": {
    "localhost:27017": 7,
    "localhost:27018": 6,
    "localhost:27019": 7
  }
}
```

Figure 6: GET /api/distribution — 20 students distributed as 7 / 6 / 7

4. Task 5: Add a Storage Node (mongo4)



```
Windows PowerShell
PS C:\Users\Admin> curl -s -X POST localhost:8080/api/nodes -H "Content-Type: application/json" -d '{"host": "mongo4", "port": 27020}' | jq "{migrated, before, after}"
{
  "migrated": 6,
  "before": {
    "localhost:27017": 7,
    "localhost:27018": 6,
    "localhost:27019": 7
  },
  "after": {
    "localhost:27017": 3,
    "localhost:27018": 5,
    "localhost:27019": 6,
    "mongo4:27020": 6
  }
}
PS C:\Users\Admin> curl -s localhost:8080/api/distribution | jq "{total, counts}"
{
  "total": 20,
  "counts": {
    "localhost:27017": 3,
    "localhost:27018": 5,
    "localhost:27019": 6,
    "mongo4:27020": 6
  }
}
PS C:\Users\Admin> |
```

Figure 7: POST /api/nodes — only ~25% of records migrate when mongo4 joins

5. Task 6: Remove a Storage Node (mongo4)

```
Administrator: Windows PowerShell
PS C:\Users\Admin> curl -s -X DELETE localhost:8080/api/nodes/mongo4/27020 | jq "{migrated, before, after}"
{
  "migrated": 6,
  "before": {
    "localhost:27017": 3,
    "localhost:27018": 5,
    "localhost:27019": 6,
    "mongo4:27020": 6
  },
  "after": {
    "localhost:27017": 7,
    "localhost:27018": 6,
    "localhost:27019": 7
  }
}
PS C:\Users\Admin> curl -s localhost:8080/api/distribution | jq "{total, counts}"
{
  "total": 20,
  "counts": {
    "localhost:27017": 7,
    "localhost:27018": 6,
    "localhost:27019": 7
  }
}
```

Figure 8: DELETE /api/nodes — removal redistributes only mongo4's records

6. Task 7–8: Modulo vs Consistent Hashing Comparison

Aspect	Modulo (hash % N)	Consistent Hashing
Lookup	hash(key) % N -> bucket	hash -> clockwise first vnode
Add node (2->3)	Every key is remapped; $\sim N/(N+1)$ of data moves	$\sim 1/(N+1)$ of keys move
Remove node (3->2)	Every key remapped	$\sim 1/N$ of keys move
Balance	Perfect balance if hash is uniform	Depends on virtual nodes; tiny variance
Dynamism	Resizing requires mass reshuffling	Nodes can join/leave with minimal, localized migration
State	None (pure function)	A ring of virtual nodes (TreeMap)

Figure 9: Migration comparison — modulo reshuffles ~all keys, consistent hashing moves $\sim 1/N$

Analysis

1. Explain the role of the hash ring, virtual nodes, and MongoDB storage node in application-level consistent hashing.

- Hash Ring: A sorted circular keypace (`TreeMap<Long, StorageNode>`) where each key's SHA-256 position decides its owner by a clockwise walk. Because the ring is ordered, lookups are $O(\log n)$ (`ceilingEntry + wrap-to-first`).
- Virtual Nodes: 150 placements per physical node spread responsibility evenly across the ring, avoiding contiguous arcs and thundering-herd migrations. `GET /api/nodes/ring` exposes the 150 vnode count per node.
- MongoDB Storage Node: A standalone mongod (`mongo1..mongo4`) holding its own `College.Student` collection. Unlike MongoDB's sharded cluster, there is no config server or mongos — the application computes the owner and connects to that single mongod directly.

2. How does consistent hashing determine the storage location of a Student record?

1. The record's RollNo (e.g. 1005) is the routing key.
2. `HashFunction.hash("1005")` = first 8 bytes of SHA-256 as an unsigned 64-bit long — the record's ring position.
3. `ConsistentHashRing.getNode("1005")` finds the smallest virtual-node position $\geq$ the hash (wrapping to the first if none), returning the owning `StorageNode`.
4. The record is written only to that node's `College.Student`. Reads use the same owner, so no fan-out is needed for a known roll number.

3. Explain what happens when a new MongoDB storage node is added.

- `POST /api/nodes { host: "mongo4", port: 27020 }` is called.
- `NodeService` inserts `mongo4`'s 150 virtual nodes; existing virtual nodes are unchanged in position.
- Every existing record's owner is re-resolved on the new ring; a record moves only if its owner changed — and only to `mongo4` (it never jumps to another pre-existing node).
- In this run 6 of 20 records (~30%, close to the ideal $1/N = 25\%$) migrated; total stayed at 20.

4. Explain what happens when an existing MongoDB storage node is removed.

`DELETE /api/nodes/mongo4/27020` is called.

`NodeService` captures `mongo4`'s records, removes its 150 virtual nodes, and re-resolves each captured record's owner on the remaining ring.

Only mongo4's records migrate; the distribution reverts exactly to the original 7 / 6 / 7 (a clean per-key inverse) with no data loss.

5. Compare modulo-based hashing with consistent hashing.

Aspect	Modulo (hash % N)	Consistent Hashing
Lookup	hash(key) % N → bucket	hash → clockwise first vnode
Add node (2→3)	Every key is remapped; ~N/(N+1) of data moves	~1/(N+1) of keys move (only the new node's range)
Remove node (3→2)	Every key is remapped; only 1/2 stays by chance	~1/N of keys move (only the removed node's data)
Balance	Perfect only if the hash is uniform and N is fixed	Depends on virtual nodes; more vnodes → better balance
Dynamism (Small Scale)	Resizing requires a mass reshuffle	Nodes join/leave with minimal, localized migration
State (Small Scale)	None (pure function)	In-memory ring of virtual nodes (TreeMap)

Note: going from 3 to 4 nodes with modulo hashing would relocate almost every record (hash % 3 → hash % 4 rarely equals), whereas consistent hashing moved only 6 of 20 records in this lab.

6. Compare application-level consistent hashing with MongoDB horizontal sharding.

Aspect	Application-Level Consistent Hashing	MongoDB Horizontal Sharding
Where the ring lives	Spring Boot process (in-memory)	mongos + config server metadata
Routing	App computes owner and picks the mongod	mongos consults the chunk map, then routes
Key	rollNo (application-chosen)	Any indexed field; hashed or ranged shard key
Balancer / migration	App migrates records on add/remove	MongoDB balancer migrates chunks automatically
Range queries (Small Scale)	App must fan out across nodes	mongos targets specific shards via chunk ranges
Data model (Small Scale)	One independent College.Student per mongod	Shared namespace across shards via mongos
Failure model (Small Scale)	App must handle a dead node (retry/redirect)	Replica-set shards survive node failure natively

MongoDB's native sharding is the more production-ready option (automatic balancing, replica-set fault tolerance, a single mongos endpoint). Application-level consistent hashing trades those operational benefits for full application control and a transparent routing layer that is ideal for learning and for workloads where the app already owns the keyspace split.

API / Command Reference & Test Results

Table 1: REST API Routing — Targeted vs Add/Remove

Scenario	Result
Student create / get / delete (by rollNo)	Routed to a single owning node via the hash ring
Add storage node (POST /api/nodes)	Only ~1/N of records migrate; total unchanged
Remove storage node (DELETE /api/nodes/...)	Only the removed node's records redistribute; total unchanged

Table 2: REST API / Commands Used

#	Command / Endpoint	Purpose
1	HashFunction.hash(key)	SHA-256 key → 64-bit ring position
2	ConsistentHashRing.getNode(key)	Clockwise lookup to the owning StorageNode
3	GET /api/distribution	Record + virtual-node counts per node
4	POST /api/nodes	Register a node; migrate only affected records
5	DELETE /api/nodes/{host}/{port}	Remove a node; redistribute its records
6	GET /api/nodes/ring	Show virtual-node counts per node

Table 3: Technology Stack

#	Technology / Tool	Purpose
1	Spring Boot 3.3 (Java 21)	Hash ring, routing logic, and REST API
2	Docker & Docker Compose	Orchestrate the standalone mongod nodes
3	MongoDB 7.0	Standalone document storage per node
4	SHA-256	Uniform key hashing onto the ring
5	Maven (mvn test)	Build and unit-test the ring logic (no Docker)

Table 4: Test Case Results

#	Test Case	Input	Expected Result	Status
1	Hash determinism	hash("1005") called twice	Same ring position	Pass
2	Even key distribution	20 roll numbers on 3 nodes	Balanced ~7 / 6 / 7	Pass
3	Add-node migration	POST /api/nodes mongo4	~25% (6/20) migrate, total 20	Pass
4	Remove-node inverse	DELETE mongo4	Reverts to 7 / 6 / 7, no data loss	Pass
5	Student create	POST /api/students	201, stored on owner node only	Pass
6	Student read	GET /api/students/{rollNo}	200, correct record	Pass
7	Student delete	DELETE /api/students/{rollNo}	204, removed from owner node	Pass
8	Total unchanged	add + remove cycle	Always 20	Pass

Learning Outcomes

- Implemented application-level consistent hashing in Spring Boot: a SHA-256 hash function, a sorted TreeMap ring, and 150 virtual nodes per physical MongoDB node.
- Routed create/read/delete of Student records to a deterministic owning node with $O(\log n)$ clockwise lookup, with no mongos or config server.
- Verified even distribution of 20 seeded records across 3 standalone nodes (7 / 6 / 7).
- Demonstrated the add-node property: only $\sim 1/N$ of records migrated when a 4th node joined.
- Demonstrated the remove-node property: only the removed node's records moved and the distribution reverted exactly, with no data loss.
- Compared modulo hashing vs consistent hashing, and application-level consistent hashing vs MongoDB horizontal sharding.